TRAVAUX PRATIQUES

Bases de Données & Modélisation des SI

TP 9 : Regrouper les données avec GROUP BY HAVING et les alias

Objectif : regrouper des lignes, calculer des indicateurs par groupe, filtrer avant et après agrégation et nommer les colonnes du résultat dans AdventureWorks sous PostgreSQL.

Le TP reprend les deux textes fournis : rappels de cours puis construction progressive des cinq cas pratiques. Les exercices finaux portent uniquement sur les notions de ces textes.

PARTIE I COMPRENDRE LES REGROUPEMENTS

I- Passer des lignes détaillées à un résultat par groupe

GROUP BY rassemble les lignes qui partagent les mêmes valeurs dans les colonnes indiquées. Une fonction d'agrégation calcule ensuite un résultat pour chaque groupe. On obtient une ligne par groupe, et non une ligne par donnée initiale.

Exemple du texte : le nom John apparaît trois fois et Sarah deux fois. Un regroupement par nom avec COUNT(*) produit John : 3 et Sarah : 2. COUNT(*) compte les lignes du groupe ; il ne compte pas des valeurs distinctes. Le nombre de lignes du résultat peut diminuer, mais il reste identique si chaque ligne constitue déjà un groupe différent.

```
SELECT color, COUNT(*)
FROM production.product
GROUP BY color;
```

Ici, chaque couleur forme un groupe. COUNT(*) fournit le nombre de produits de cette couleur. Pour vérifier le résultat, observez les valeurs répétées dans les données de départ et les effectifs obtenus après regroupement.

II- Choisir une fonction d'agrégation

COUNT(*) compte les lignes. SUM(colonne) calcule une somme, AVG(colonne) une moyenne, MIN(colonne) la plus petite valeur et MAX(colonne) la plus grande. Pour les calculs sur une colonne, précisez le nom de cette colonne : AVG(*) ne remplace pas AVG(listprice).

Exemple de quantités vendu dans le texte : pour John, les quantités sont 10, 20 et 30. Le maximum vaut 30, la somme 60 et la moyenne 20. Pour Sarah, les quantités 15 et 5 donnent un maximum de 15, une somme de 20 et une moyenne de 10. Ces calculs concernent les lignes de chaque vendeur ; ils ne regroupent pas automatiquement les quantités par produit.

```
SELECT productsubcategoryid,
       AVG(listprice), MIN(listprice), MAX(listprice)
FROM production.product
GROUP BY productsubcategoryid;
```

Chaque sous-catégorie possède ici ses trois indicateurs. Le prix minimum, le prix maximum et le prix moyen ne désignent pas nécessairement le prix d'un même produit. Une moyenne peut avoir de nombreuses décimales ; sa mise en forme n'est pas abordée dans ces textes.

III- Regrouper sur plusieurs colonnes

Avec deux colonnes, chaque combinaison constitue un groupe. Dans l'exemple noms et villes, John Paris apparaît deux fois, John Marseille une fois, Sarah Lyon une fois et Sarah Paris une fois. Le regroupement par nom seul donnerait un autre résultat.

```
SELECT productsubcategoryid, color, COUNT(*)
FROM production.product
GROUP BY productsubcategoryid, color;
```

Le couple sous-catégorie et couleur identifie un groupe. Inverser ces deux colonnes dans GROUP BY ne change pas les combinaisons formées. GROUP BY ne garantit aucun ordre d'affichage : utilisez ORDER BY lorsque vous souhaitez trier le résultat.

Pour les requêtes de ce TP, reprenez dans GROUP BY toutes les colonnes affichées sans fonction d'agrégation. Les expressions COUNT(*), AVG(listprice), MIN(listprice) et MAX(listprice) ne sont pas à recopier dans GROUP BY. La colonne groupée n'a pas à être obligatoirement affichée, mais son affichage permet ici d'identifier chaque résultat.

```
-- Regroupement coherent avec les colonnes affichees
SELECT productsubcategoryid, color, COUNT(*)
FROM production.product
GROUP BY productsubcategoryid, color;
```

Si color est affichée sans être agrégée alors que seul productsubcategoryid figure dans GROUP BY, une sous-catégorie pourrait contenir plusieurs couleurs : le résultat demandé serait ambigu. Dans cet exemple, ajoutez color au regroupement.

IV- Comprendre les valeurs NULL dans les groupes

Les lignes dont la clé de regroupement est NULL sont réunies dans un même groupe pour cette clé. COUNT(*) compte aussi ces lignes. Un groupe de couleur NULL signifie que la couleur est absente ; il ne signifie pas qu'aucun produit n'a été trouvé.

```
SELECT color, COUNT(*)
FROM production.product
GROUP BY color;
```

Avec deux clés, la combinaison (NULL, NULL) constitue un groupe, distinct d'une combinaison où seule l'une des clés est NULL. Pour écarter des lignes sans identifiant de vendeur, le texte propose un filtre IS NOT NULL avant le regroupement.

V- Filtrer les groupes avec HAVING

WHERE sélectionne les lignes qui entrent dans le calcul. HAVING sélectionne les groupes à partir du résultat du calcul. Ainsi, pour conserver uniquement les sous-catégories comptant plus de dix produits, il faut compter avant de vérifier le seuil.

```
SELECT productsubcategoryid, COUNT(*)
FROM production.product
GROUP BY productsubcategoryid
HAVING COUNT(*) > 10;
```

Le seuil > 10 signifie strictement plus de dix. Pour « au moins dix », utilisez >= 10. Cette distinction corrige les formulations contradictoires de la transcription. De même, au moins 100 se traduit par >= 100 et plus de 100 par > 100.

Le texte évoque aussi la somme des ventes par catégorie avec un seuil de 10 000. Le principe est identique : calculer SUM sur la colonne concernée, puis écrire HAVING SUM(colonne) > 10000. Ne supposez pas que la table production.product contient une colonne de ventes : cet exemple est conceptuel.

VI- Nommer une colonne du résultat avec AS

Un alias est un nom donné à une colonne du résultat. Il améliore la lecture sans renommer la colonne stockée ni modifier la table. AS s'écrit après la colonne ou l'expression à nommer.

```
SELECT productsubcategoryid AS sous_categorie,
       COUNT(*) AS product_count
FROM production.product
GROUP BY productsubcategoryid
ORDER BY product_count DESC;
```

Dans PostgreSQL, l'alias product_count peut être utilisé dans ORDER BY. Il ne peut pas remplacer COUNT(*) dans HAVING, ni être utilisé comme nouvelle colonne dans WHERE ou dans une autre expression du même SELECT. Dans HAVING, répétez l'expression d'agrégation. Cette règle corrige le passage du texte qui autorise trop largement les alias dans HAVING.

```
SELECT productsubcategoryid, COUNT(*) AS product_count
FROM production.product
GROUP BY productsubcategoryid
HAVING COUNT(*) > 10
ORDER BY product_count DESC;
```

VII- Respecter l'ordre des clauses

L'ordre d'écriture utilisé dans ces textes est SELECT, FROM, WHERE, GROUP BY, HAVING puis ORDER BY. Les clauses facultatives absentes sont simplement omises. Pour comprendre le calcul, raisonnez ainsi : sélectionner les lignes avec WHERE, former les groupes et les indicateurs, retenir les groupes avec HAVING, puis trier le résultat.

```
SELECT salespersonid, COUNT(*) AS order_count
FROM sales.salesorderheader
WHERE orderdate >= '2004-01-01'
  AND salespersonid IS NOT NULL
GROUP BY salespersonid
HAVING COUNT(*) > 100
ORDER BY order_count DESC;
```

PARTIE II MANIPULATIONS GUIDÉES DANS ADVENTUREWORKS

VIII- Compter les commandes par représentant des ventes

1. Ouvrez Query Tool sur AdventureWorks et enregistrez TP9_Nom_Prenom.sql. Consultez la table indiquée dans le texte.

```
SELECT *
FROM sales.salesorderheader;
```

2. Repérez salesorderid, orderdate et salespersonid. Dans cette table, une ligne représente une commande. salespersonid est l'identifiant du représentant associé ; il peut être absent.

3. Conservez l'identifiant du vendeur et comptez les commandes pour chaque groupe.

```
SELECT salespersonid, COUNT(*)
FROM sales.salesorderheader
GROUP BY salespersonid;
```

4. Donnez au comptage le nom order_count, puis triez cet indicateur par ordre décroissant.

```
SELECT salespersonid, COUNT(*) AS order_count
FROM sales.salesorderheader
GROUP BY salespersonid
ORDER BY order_count DESC;
```

Le résultat contient un total de commandes par identifiant, et éventuellement un groupe d'identifiant NULL. Ce groupe ne représente pas un vendeur identifié ; il réunit les commandes sans identifiant renseigné. Ne concluez pas automatiquement à une erreur : l'interprétation relève des données métier.

IX- Calculer les prix moyen minimum et maximum par sous-catégorie

5. Consultez production.product et repérez productsubcategoryid et listprice.

```
SELECT *
FROM production.product;
```

6. Commencez avec la seule moyenne par sous-catégorie et donnez-lui un alias.

```
SELECT productsubcategoryid, AVG(listprice) AS avg_price
FROM production.product
GROUP BY productsubcategoryid;
```

7. Ajoutez le prix maximum et le prix minimum. Vérifiez que les noms de colonnes sont correctement orthographiés.

```
SELECT productsubcategoryid,
       AVG(listprice) AS avg_price,
       MAX(listprice) AS max_price,
       MIN(listprice) AS min_price
FROM production.product
GROUP BY productsubcategoryid;
```

8. Triez le résultat sur avg_price par ordre décroissant, comme dans la démonstration.

```
SELECT productsubcategoryid,
       AVG(listprice) AS avg_price,
       MAX(listprice) AS max_price,
       MIN(listprice) AS min_price
FROM production.product
GROUP BY productsubcategoryid
ORDER BY avg_price DESC;
```

Contrôle : pour des prix renseignés, la moyenne d'un groupe doit être comprise entre son minimum et son maximum. Conservez les décimales retournées : aucune fonction d'arrondi ou de troncature n'est ajoutée dans ce TP.

X- Compter les produits par combinaison de sous-catégorie et modèle

9. Affichez les deux colonnes de regroupement avant de calculer les effectifs.

```
SELECT productsubcategoryid, productmodelid
FROM production.product;
```

10. Formez un groupe par combinaison, puis comptez ses produits.

```
SELECT productsubcategoryid, productmodelid,
       COUNT(*) AS product_count
FROM production.product
GROUP BY productsubcategoryid, productmodelid
ORDER BY product_count DESC;
```

11. Repérez une combinaison répétée et, si elle existe, la combinaison dont les deux identifiants sont NULL. Lisez chaque ligne comme un couple d'identifiants et son effectif.

Les 209 lignes pour la combinaison absente ou les effectifs de douze produits cités dans le texte sont des observations de la démonstration. Relevez les valeurs de votre sauvegarde sans imposer ces nombres. Les jointures permettant de retrouver les libellés ne font pas partie de ce TP.

XI- Conserver les représentants ayant traité au moins 100 commandes

12. Reprenez le comptage par vendeur, puis ajoutez HAVING avant ORDER BY.

```
SELECT salespersonid, COUNT(*) AS order_count
FROM sales.salesorderheader
GROUP BY salespersonid
HAVING COUNT(*) >= 100
ORDER BY order_count DESC;
```

13. Vérifiez que chaque effectif affiché atteint 100. Expliquez pourquoi HAVING order_count >= 100 est incorrect dans PostgreSQL alors que ORDER BY order_count fonctionne.

La consigne de ce cas dit « au moins 100 » : la valeur 100 doit être acceptée. La démonstration emploie parfois > 100 ; ce filtre correspond à « plus de 100 ». Un groupe NULL peut encore satisfaire HAVING : ce seuil ne supprime pas les identifiants absents.

XII- Combiner un filtre de date et un filtre sur le nombre de commandes

14. Conservez les commandes à partir du 1er janvier 2004 inclus, puis comptez par vendeur et gardez les groupes ayant plus de 100 commandes.

```
SELECT salespersonid, COUNT(*) AS order_count
FROM sales.salesorderheader
WHERE orderdate >= '2004-01-01'
GROUP BY salespersonid
HAVING COUNT(*) > 100
ORDER BY order_count DESC;
```

Le texte emploie « après le 1er janvier » mais montre >=. Nous retenons ici la règle explicite « à partir du 1er janvier inclus ». COUNT(*) porte uniquement sur les commandes conservées par WHERE, pas sur toutes les commandes historiques du vendeur.

15. Ajoutez le filtre IS NOT NULL demandé dans la dernière étape de la démonstration.

```
SELECT salespersonid, COUNT(*) AS order_count
FROM sales.salesorderheader
WHERE orderdate >= '2004-01-01'
  AND salespersonid IS NOT NULL
GROUP BY salespersonid
HAVING COUNT(*) > 100
ORDER BY order_count DESC;
```

16. Comparez les résultats avant et après l'exclusion des identifiants absents. Expliquez séparément le rôle de la date, du test IS NOT NULL, du regroupement, de HAVING et du tri.

Construisez les requêtes progressivement : consulter la table, identifier les colonnes, regrouper, calculer, nommer les résultats, puis ajouter les filtres et le tri. Les noms d'objets doivent correspondre à ceux de votre sauvegarde.

Travail pratique en classe

Travail pratique en classe

1. Combien de produits y a-t-il dans chaque sous-catégorie ? Comptez le nombre de produits pour chaque sous-catégorie dans la table Production.Product
2. Quel est le poids minimum, maximum et moyen des produits pour chaque sous-catégorie ? Trouver le poids (weigth) minimum, maximum et moyen des produits pour chaque catégorie dans la table Production.Product.
3. Combien de produits y a-t-il pour chaque combinaison de model (productmodelid) et de coleur (color) ?
4. Quelles sous-catégories (ProductsubCategoryID) de produits ont plus de 10 produits (ProductID) ?
5. Quelles sous-catégories (ProductsubCategoryID) de produits ont plus de 10 produits (ProductID) dont le prix (listprice) est supérieur à 100$?

Travail pratique à rendre

Exercice 1- Expliquer les agrégations

1. Expliquez ce que compte COUNT(*) dans un groupe.
2. Reprenez les quantités de John et Sarah du cours et calculez SUM, AVG, MIN et MAX pour chaque vendeur.
3. Pourquoi GROUP BY sur nom et ville diffère-t-il de GROUP BY sur nom seul ?
4. Comment un groupe de clé NULL est-il traité avec COUNT(*) ?

Exercice 2 Refaire les calculs de la démonstration

Q1. Compter les commandes par salespersonid avec l'alias order_count, trié par ordre décroissant.
Q2. Afficher le prix moyen, minimum et maximum par productsubcategoryid, avec les alias étudiés et un tri décroissant sur avg_price.
Q3. Compter les produits par combinaison de productsubcategoryid et productmodelid, avec l'alias product_count et un tri décroissant sur cet effectif.

Exercice 3 Appliquer les filtres du texte

Q4. Conserver les groupes ayant au moins 100 commandes.
Q5. Conserver les représentants identifiés ayant plus de 100 commandes à partir du 1er janvier 2004 inclus.
Expliquez pour Q5 quelles lignes sont éliminées par WHERE et quels groupes sont éliminés par HAVING.

Exercice 4- Corriger les erreurs rencontrées

```
-- A : compter par sous-categorie et couleur
SELECT productsubcategoryid, color, COUNT(*)
FROM production.product
GROUP BY productsubcategoryid;

-- B : au moins 100 commandes par vendeur
SELECT salespersonid, COUNT(*) AS order_count
FROM sales.salesorderheader
GROUP BY salespersonid
HAVING order_count >= 100;

-- C : filtrer le nombre de commandes apres regroupement
SELECT salespersonid, COUNT(*) AS order_count
FROM sales.salesorderheader
WHERE COUNT(*) > 100
GROUP BY salespersonid;
```

Pour chaque cas, identifiez le défaut et écrivez une correction. Employez uniquement GROUP BY, les agrégations, AS, WHERE, HAVING et ORDER BY étudiés dans les textes.

Remise

Remettez TP9_Nom_Prenom.sql commenté et un court compte rendu contenant les réponses, les corrections et les captures de Q2 et Q5. Relevez les effectifs de votre sauvegarde.